

DupliSense AI: Intelligent Project Similarity Detection and Software Reuse Recommendation System

Gokul Anand B
Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Chennai, India
230701093@rajalakshmi.edu.in

Gayathri V R
Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Chennai, India
23070190@rajalakshmi.edu.in

Jyoshna K
Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Chennai, India
230701132@rajalakshmi.edu.in

Harshita M A
Department of Computer Science and
Engineering
Rajalakshmi Engineering College
Chennai, India
230701112@rajalakshmi.edu.in

N Duraimurugan
Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Chennai, India
duraimurugan.n@rajalakshmi.edu.in

Abstract—Software organizations and academic institutions may maintain multiple projects that address related problems or use overlapping technologies. Identifying such overlap manually can be time-consuming and may lead to repeated development efforts and underutilization of existing software resources. DupliSense AI is a web-based platform designed to identify similar software projects and support software reuse. The system collects project title, description, objectives, problem statement, and technology stack, and transforms the combined project information into numerical representations using Term Frequency-Inverse Document Frequency (TF-IDF). Cosine Similarity is then used to compare project representations and classify project pairs into High, Medium, Partial, and Low similarity levels. Based on significant similarity, the platform presents possible reuse opportunities involving code, APIs, database structures, user interface components, documentation, and team expertise. The system also implements Role-Based Access Control (RBAC) for Developer, Manager, and Admin roles. The prototype uses React and Vite for the frontend, Django REST Framework for backend services, SQLite for persistence, and scikit-learn for NLP processing. The resulting workflow provides a centralized mechanism for project comparison, similarity visualization, and software reuse support.

Index Terms—DupliSense AI, project similarity detection, natural language processing, TF-IDF, cosine similarity, software reuse, role-based access control, Django REST Framework, React

I. INTRODUCTION

Software development organizations and academic institutions often maintain multiple projects that may address related problems or use overlapping technologies. When project information is distributed across different teams or repositories, identifying such overlap manually can be difficult and time-consuming. Research on duplicate software reports has shown that information-retrieval and text-analysis techniques can be

used to identify related reports and potentially duplicate issues [1]. Similarity analysis has therefore become useful in software engineering scenarios where large collections of textual project or service information must be compared.

Natural Language Processing (NLP) provides methods for representing and processing software-related textual information [2]. Classical text-similarity approaches can represent documents as vectors and compare their relationships using numerical similarity measures [4]. These approaches provide a practical foundation for project-level similarity analysis.

DupliSense AI addresses this requirement through a centralized web platform that registers project information, performs automated similarity analysis, classifies the resulting similarity level, and presents possible software reuse opportunities. The platform is intended to support developers in discovering related projects before reimplementing functionality that may already exist.

The major contributions of the proposed system are:

- Automated identification of potentially similar software projects from structured project descriptions.
- NLP-based project comparison using TF-IDF feature representation and Cosine Similarity.
- Similarity classification into High, Medium, Partial, and Low categories for easier interpretation.
- Reuse-oriented recommendations covering code, APIs, database structures, interface components, documentation, and relevant expertise.
- Role-based access for Developers, Managers, and Administrators, supporting controlled project and recommendation workflows.

II. RELATED WORK

Duplicate and related-item detection has been studied in software engineering using information retrieval, topic modeling, and other text-based approaches. Nguyen *et al.* investigated duplicate bug report detection by combining information retrieval and topic modeling techniques [1]. Such work demonstrates the usefulness of textual similarity for finding related software artifacts.

NLP has also been surveyed specifically in the context of software engineering, covering applications in requirements, source code, documentation, and other software artifacts [2]. Text similarity methods provide additional foundations for comparing documents using lexical and vector-based representations [4]. These techniques are relevant to project descriptions because project information is primarily represented as textual content.

More recent semantic approaches use sentence embeddings. Reimers and Gurevych introduced Sentence-BERT (SBERT), which produces sentence embeddings that can be compared efficiently using similarity measures such as cosine similarity [5]. DupliSense AI currently uses the more lightweight TF-IDF and Cosine Similarity pipeline for its core project comparison workflow, while embedding-based approaches are considered as a future extension.

Similarity detection has also been explored in software repositories and service-desk environments, where identifying related requests or projects can support knowledge reuse and reduce repeated analysis [3], [8]. The proposed platform extends this general idea by combining similarity detection with reuse recommendations and role-based project governance.

Role-Based Access Control provides a structured mechanism for assigning permissions according to user roles [6]. DupliSense AI applies this principle to separate Developer, Manager, and Admin capabilities. The platform also exposes backend functionality through REST APIs, which supports modular communication between the frontend and backend components [9].

III. PROPOSED METHODOLOGY

The proposed DupliSense AI system follows a workflow consisting of project registration, text preparation, feature extraction, similarity calculation, classification, reuse recommendation, and result visualization.

Project information such as project title, description, objectives, problem statement, and technology stack is collected through the web application. The information is stored in the database and prepared for NLP processing.

The relevant textual fields are combined into a unified representation. TF-IDF vectorization is then applied to transform the project text into numerical vectors. Cosine Similarity is used to compare project vectors. The resulting similarity score is mapped to a similarity category. When significant similarity is detected, the system displays possible reuse opportunities for review.

A. System Architecture

The system consists of a React and Vite frontend, Django REST Framework backend, SQLite database, and NLP processing components. The frontend communicates with backend services through REST APIs. REST-based communication supports a separation between presentation, application logic, and persistence layers [9].

The frontend provides interfaces for project submission, similarity results, reuse recommendations, dashboards, and role-specific operations. The backend handles API requests, authentication, project processing, similarity computation, recommendation logic, and database communication.

The database stores project information, user information, similarity results, and recommendation-related data. The resulting workflow provides a centralized environment for project comparison and reuse support.

B. Project Data Collection

Project information is collected through the web application. The input contains the project title, description, objectives, problem statement, and technology stack. These fields provide the textual evidence used for subsequent comparison.

Centralizing project information allows the system to compare multiple projects using a common representation rather than requiring users to manually inspect separate project records.

C. Text Preprocessing

The collected project information is combined into a unified text representation before similarity analysis. The processing stage prepares the input for feature extraction and helps maintain a consistent format across project records.

The quality of the similarity result depends on the quality and completeness of the supplied project descriptions. Projects with sparse or highly inconsistent descriptions may produce less representative textual comparisons.

D. TF-IDF Feature Extraction

Term Frequency–Inverse Document Frequency (TF-IDF) represents the importance of terms within a document collection. It is a standard vector-space representation for textual information and is commonly used in information-retrieval applications [4].

The TF-IDF value of a term t in document d can be represented as

$$TFIDF(t, d) = TF(t, d) \times IDF(t) \quad (1)$$

where $TF(t, d)$ represents the frequency of term t in document d , and $IDF(t)$ represents the inverse document frequency of the term.

For DupliSense AI, the resulting vector represents the textual characteristics of a project and provides the input for similarity calculation.

E. Cosine Similarity

Cosine Similarity measures the angular similarity between two numerical vectors. It is suitable for comparing document representations because the direction of a vector can be compared independently of its magnitude [4].

For two project vectors A and B , the similarity is calculated as

$$\text{Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|} \quad (2)$$

where A and B represent the TF-IDF vectors of two projects.

A larger similarity value indicates that the two vector representations are more closely related, while a smaller value indicates less textual similarity. The score is subsequently mapped to one of the system's similarity categories.

F. Similarity Classification

The calculated similarity score is converted into an interpretable classification. The system provides four categories: High, Medium, Partial, and Low similarity.

These categories are intended as decision-support indicators rather than absolute proof that two projects are functionally identical. A high textual similarity can indicate that projects should be reviewed for potential reuse, while a low textual similarity does not guarantee that the projects have no reusable components.

G. Reuse Recommendation

When significant project similarity is detected, DupliSense AI presents possible reuse opportunities. The recommendation categories include reusable source code, APIs, database structures, user-interface components, documentation, and relevant team expertise.

Software reuse aims to avoid unnecessary redevelopment by identifying existing assets that can potentially be incorporated into another project. The system therefore connects the similarity result with a reuse-oriented workflow rather than stopping at document comparison.

Managers can review recommendations before reuse access is enabled, providing a governance step between automated similarity detection and actual resource reuse.

H. Role-Based Access Control

The system incorporates Role-Based Access Control (RBAC), a model in which permissions are associated with roles and users obtain access through those roles [6].

The primary roles are:

- **Developer:** submits and maintains project information, reviews similarity results, and views reuse recommendations.
- **Manager:** reviews reuse recommendations and can approve or reject proposed reuse access.
- **Admin:** monitors system activities, project statistics, reuse patterns, and analytics.

This role-based structure separates project submission, managerial review, and system administration responsibilities.

IV. MATHEMATICAL MODEL

Let a collection of projects be represented as

$$P = \{P_1, P_2, \dots, P_n\} \quad (3)$$

where P_i represents an individual project.

Each project is transformed into a TF-IDF vector:

$$P_i = [w_1, w_2, \dots, w_m] \quad (4)$$

where w_j represents the TF-IDF weight of the corresponding term.

For two projects P_i and P_j , the similarity is calculated using

$$S(P_i, P_j) = \frac{P_i \cdot P_j}{\|P_i\| \|P_j\|} \quad (5)$$

where $S(P_i, P_j)$ represents the Cosine Similarity score.

The resulting score is used for similarity classification and recommendation generation.

V. SOFTWARE REQUIREMENTS

DupliSense AI is implemented as a software-only web application. The major technologies used in the prototype are described below.

A. Frontend Technologies

The frontend is developed using React and Vite. React is used to build the interactive user interface, while Vite provides the development and build environment.

The frontend contains project submission, similarity results, recommendation, dashboard, and role-specific views.

B. Backend Technologies

The backend is implemented using Django REST Framework. It provides RESTful services for communication between the frontend and application logic.

The backend manages authentication, project data, similarity processing, recommendation generation, RBAC, and database operations. The REST interface follows the general architectural approach of separating client and server responsibilities [9].

C. Database

SQLite is used for data persistence in the prototype. The database stores project details, user information, similarity results, recommendations, and related application data.

D. NLP and Machine Learning Technologies

The similarity-processing component uses scikit-learn functionality for TF-IDF vectorization and numerical similarity computation. Scikit-learn provides a widely used Python machine-learning toolkit that includes text-processing utilities [2].

The current core pipeline uses TF-IDF and Cosine Similarity. Sentence embedding approaches such as SBERT are considered for future semantic similarity extensions [5].

VI. SYSTEM IMPLEMENTATION

The implementation is divided into project management, similarity detection, recommendation, authentication, role-based access, and analytics components.

A. Project Management Module

The project management module allows users to register and maintain project information. Project title, description, objectives, problem statement, and technology stack are stored for analysis.

B. Similarity Detection Module

The similarity detection module combines relevant project text and converts it into TF-IDF vectors. Cosine Similarity is then applied to calculate pairwise project similarity.

The calculated results are stored and displayed through the similarity results interface. The general concept follows earlier software engineering work in which textual representations were used to identify duplicate or related artifacts [1], [3].

C. Recommendation Module

The recommendation module analyzes significant similarity results and presents possible software reuse opportunities. The recommendations may include code, APIs, database structures, interface components, documentation, and team expertise.

The purpose is to connect similarity analysis with practical reuse decisions instead of treating similarity as an isolated numerical output. Software reuse has long been studied as a means of improving the use of existing software assets [10].

D. Analytics Module

The analytics module provides information related to project similarity, reuse activity, project statistics, and potential resource savings.

Dashboards and visual representations are used to make similarity and reuse information easier for users and managers to inspect.

VII. RESULTS AND DISCUSSION

The developed DupliSense AI prototype provides an automated workflow for comparing project descriptions and presenting possible reuse opportunities. A project can be submitted through the developer workspace, compared against existing project information, and displayed with a similarity classification and corresponding reuse information.

The interface also supports a manager-review stage. When a reuse recommendation is approved, the system can expose the verified repository or source-code access associated with the reuse workflow. This connects similarity detection with controlled reuse rather than providing unrestricted access.

The screenshot in Fig. 1 illustrates a similarity result page in the DupliSense AI interface. It shows a high-similarity classification, an overlap percentage displayed by the application, matched technologies, matched concepts, and a manager-approved repository/source-code access section.

The categories in Table I provide an interpretable view of the numerical similarity result. They should be considered

TABLE I
PROJECT SIMILARITY CLASSIFICATION

Similarity Level	Description
High	Strong project similarity
Medium	Significant similarity
Partial	Limited similarity
Low	Minimal similarity

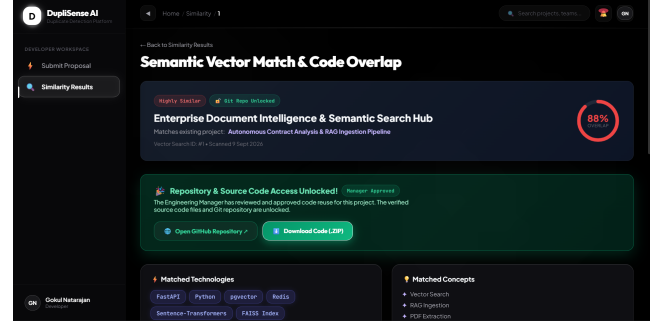


Fig. 1. DupliSense AI similarity analysis and software reuse recommendation interface. The interface displays similarity classification, matched technologies and concepts, and manager-approved repository access.

screening categories for further review rather than validated performance classes.

No accuracy, precision, recall, F1-score, or other benchmark performance value is claimed in this paper because a validated experimental dataset and corresponding evaluation results are not part of the present prototype description.

VIII. LIMITATIONS

The effectiveness of the similarity detection process depends on the quality and completeness of the project information supplied by users.

The current core approach primarily relies on textual TF-IDF representation. Consequently, terminology differences, synonyms, paraphrasing, and differences in writing style may affect the similarity score. More semantic representation methods can potentially address some of these limitations [5].

The system's reuse recommendations indicate potential reusable resources; they do not by themselves establish that a resource is technically compatible, secure, licensed for reuse, or suitable for a particular project. Human review remains part of the proposed workflow.

IX. FUTURE WORK

Future development can improve semantic project comparison by incorporating sentence-embedding or language-model-based techniques. SBERT provides one established approach for producing semantically meaningful sentence embeddings that can be compared using cosine similarity [5].

The platform can also be extended with larger project repositories, repository-level integration, improved reusable-component identification, richer team expertise matching, and advanced analytics.

Additional work can evaluate the system using a labelled project similarity dataset and report standard information-retrieval metrics. Such an evaluation would provide quantitative evidence for comparing different similarity methods.

X. CONCLUSION

DupliSense AI provides a centralized approach for identifying similar software projects and supporting software reuse. By combining NLP-based text representation, TF-IDF feature extraction, and Cosine Similarity, the platform can compare project information and classify potential overlap.

The reuse recommendation component connects similarity results with possible software assets such as code, APIs, database structures, interface components, documentation, and team expertise. RBAC provides controlled access for Developers, Managers, and Administrators [6].

The integration of project registration, similarity analysis, recommendation, managerial review, and analytics provides a structured workflow for discovering potentially overlapping projects and encouraging reuse of existing development resources.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to **Mr. Duriamurugan N, Faculty Coordinator**, Department of Computer Science and Engineering, Rajalakshmi Engineering College, for his guidance, support, and valuable feedback throughout the development of the DupliSense AI project.

The authors also thank the Department of Computer Science and Engineering and Rajalakshmi Engineering College for providing the academic environment and resources required to carry out this work.

REFERENCES

- [1] A. T. Nguyen, T. T. Nguyen, T. N. Nguyen, D. Lo, and C. Sun, "Duplicate Bug Report Detection with a Combination of Information Retrieval and Topic Modeling," in *Proc. IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2012.
- [2] X. Xia, D. Lo, X. Wang, and B. Zhou, "A Survey on Natural Language Processing for Software Engineering," *IEEE Transactions on Software Engineering*, 2018.
- [3] J. He, M. F. Zibran, and C. K. Roy, "Detecting Duplicate Requests in Online Software Service Desks," in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, 2019.
- [4] W. H. Gomaa and A. A. Fahmy, "A Survey of Text Similarity Approaches," *International Journal of Computer Applications*, vol. 68, no. 13, 2013.
- [5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing and the 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992, 2019.
- [6] R. Sandhu, E. Coyne, H. Feinstein, and C. Youman, "Role-Based Access Control Models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.
- [7] H. Huang, "A Comparison of Document Similarity Methods," in *Proc. Int. Conf. Computational Linguistics*, 2020.
- [8] M. Ortu, G. Destefanis, A. Ferrara, and M. Marchesi, "Automatic Identification of Similar Projects in Software Repositories," in *Proc. IEEE Int. Conf. Software Analysis, Evolution and Reengineering (SANER)*, 2020.
- [9] C. Pautasso, O. Zimmermann, and F. Leymann, "REST API Design Best Practices," *IEEE Internet Computing*, 2016.
- [10] C. W. Krueger, "Software Reuse: A Comprehensive Survey," *ACM Computing Surveys*, 2021.